

Adaptive DC Motor Speed Control with Drivetrain Compliance

Design physics-aware speed control for a DC motor driving a load through a flexible shaft. Keeping the reflected load inertia J_L symbolic exposes how drivetrain compliance shifts the torsional resonance, then produces state-feedback gains $K(J_L)$ that adapt across traction and vehicle-load conditions without re-tuning.

Workflow: derive the symbolic 4-state motor-shaft-load model $\rightarrow$ substitute motor and shaft constants while sweeping $J_L \rightarrow$ analyze open-loop pole migration and resonance shift $\rightarrow$ place closed-loop poles symbolically for $K(J_L) \rightarrow$ generate adaptive gain and reference-scaling functions with `matlabFunction` $\rightarrow$ validate speed tracking across lost-traction and heavy-load cases

Requires: Symbolic Math Toolbox™, Control System Toolbox™

Copyright 2026 The MathWorks, Inc.

1. Symbolic System Modeling

Four states model the motor-shaft-load drivetrain: motor speed ω_m , armature current i , shaft twist angle $\Delta\theta$, and load speed ω_L . Input: armature voltage V . Output: load speed ω_L . The flexible shaft coupling (stiffness K_s , damping C_s) between motor and load creates a torsional resonance, which is the key dynamic that shifts with J_L .

Governing equations:

- Motor: $J_m \dot{\omega}_m = K_m i - b_m \omega_m - K_s \Delta\theta - C_s(\omega_m - \omega_L)$
- Electrical: $L \dot{i} = V - Ri - K_m \omega_m$
- Shaft: $\dot{\Delta\theta} = \omega_m - \omega_L$
- Load: $J_L \dot{\omega}_L = K_s \Delta\theta + C_s(\omega_m - \omega_L) - b_L \omega_L$

Physical parameters: J_m - motor inertia [$\text{kg}\cdot\text{m}^2$], J_L - load inertia, b_m/b_L - friction [$\text{N}\cdot\text{m}\cdot\text{s}$], K_m - motor constant [$\text{V}/(\text{rad/s})$], R - resistance [Ω], L - inductance [H], K_s - shaft stiffness [$\text{N}\cdot\text{m}/\text{rad}$], C_s - shaft damping [$\text{N}\cdot\text{m}\cdot\text{s}$].

```
syms J_L J_m b_m b_L Km R L K_s C_s positive
A = [(-b_m - C_s)/J_m, Km/J_m, -K_s/J_m, C_s/J_m;
     -Km/L, -R/L, 0, 0];
```

```

    1,      0,      0,      -1;
    C_s/J_L, 0,      K_s/J_L, (-b_L - C_s)/J_L];
B = [0; 1/L; 0; 0];
C = [0 0 0 1];
D = 0;
disp('State-space model (symbolic):')

```

State-space model (symbolic):

A =

A

A =

$$A = \begin{bmatrix} -\frac{C_s + b_m}{J_m} & \frac{K_m}{J_m} & -\frac{K_s}{J_m} & \frac{C_s}{J_m} \\ -\frac{K_m}{L} & -\frac{R}{L} & 0 & 0 \\ 1 & 0 & 0 & -1 \\ \frac{C_s}{J_L} & 0 & \frac{K_s}{J_L} & -\frac{C_s + b_L}{J_L} \end{bmatrix}$$

B =

B

B =

$$B = \begin{bmatrix} 0 \\ \frac{1}{L} \\ 0 \\ 0 \end{bmatrix}$$

2. Open-Loop Dynamics: How Load Inertia Affects the Plant

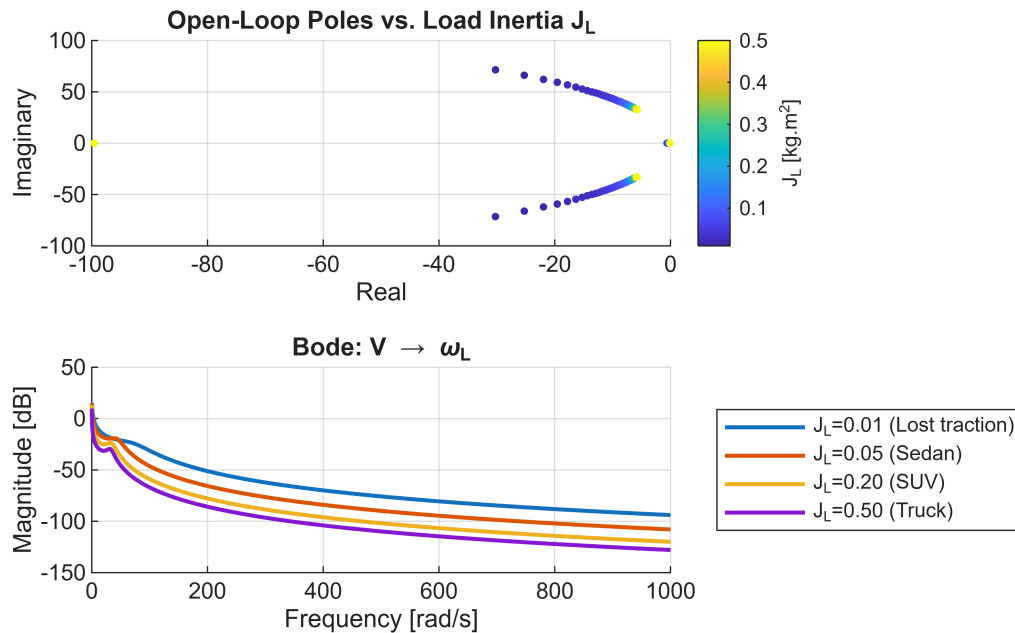
Substitute known motor and shaft constants but keep J_L symbolic. Sweeping J_L reveals two key effects: the plant poles migrate in the complex plane, and the torsional resonance peak shifts in the frequency response. This is the core problem that motivates adaptive control.

```
J_m_val = 0.05; b_m_val = 0.01; b_L_val = 0.005;
Km_val = 0.1; R_val = 0.5; L_val = 0.005;
K_s_val = 50; C_s_val = 0.5;
motor_params = [J_m b_m b_L Km R L K_s C_s];
motor_values = [J_m_val b_m_val b_L_val Km_val R_val L_val K_s_val C_s_val];
A_motor = subs(A, motor_params, motor_values);
B_Val = double(subs(B, motor_params, motor_values));
C_Val = double(C);
D_Val = double(D);
% Sweep J_L and compute eigenvalues numerically
J_L_range = linspace(0.01, 0.5, 200);
n_pts = length(J_L_range);
all_re = zeros(4*n_pts, 1);
all_im = zeros(4*n_pts, 1);
all_JL = zeros(4*n_pts, 1);
for idx = 1:n_pts
    A_n = double(subs(A_motor, J_L, J_L_range(idx)));
    ev = eig(A_n);
    rows = (4*(idx-1)+1):(4*idx);
    all_re(rows) = real(ev);
    all_im(rows) = imag(ev);
    all_JL(rows) = J_L_range(idx);
end
tiledlayout(2, 1)
nexttile
scatter(all_re, all_im, 8, all_JL, 'filled')
xlabel('Real'); ylabel('Imaginary')
title('Open-Loop Poles vs. Load Inertia J_L')
colormap(parula); clim([0.01 0.5])
cb = colorbar; cb.Label.String = 'J_L [kg.m^2]';
grid on
nexttile
hold on
J_L_bode = [0.01, 0.05, 0.2, 0.5];
```

```

labels_bode = ["Lost traction", "Sedan", "SUV", "Truck"];
colors_bode = lines(length(J_L_bode));
w = logspace(-1, 3, 500);
for idx = 1:length(J_L_bode)
    A_n = double(subs(A_motor, J_L, J_L_bode(idx)));
    sys_ol = ss(A_n, B_Val, C_Val, D_Val);
    [mag, ~, ~] = bode(sys_ol, w);
    semilogx(w, 20*log10(squeeze(mag)), 'Color', colors_bode(idx,:), ...
        'LineWidth', 1.5, 'DisplayName', sprintf('J_L=%.2f (%s)', J_L_bode(idx), labels_bode(idx)));
end
xlabel('Frequency [rad/s]'); ylabel('Magnitude [dB]')
title('Bode: V \rightarrow \omega_L')
legend('Location', 'eastoutside')
grid on

```



3. State Feedback Gains as Functions of J_L

Design a state feedback controller: $u = -Kx + \bar{N}r$. The desired closed-loop poles define the performance spec. Because the system matrix A contains symbolic J_L , the four gains $K = [k_1, k_2, k_3, k_4]$ come out as functions of J_L .

Deployment note: This example assumes full-state feedback. In production hardware, shaft twist angle $\Delta\theta$ is usually estimated rather than measured, so an observer such as a Luenberger observer or EKF would provide $\hat{x}$ from motor current, motor speed, and wheel/load speed sensor data.

```
n = size(A, 1);
Co = [B, A*B, A^2*B, A^3*B];
det_Co = simplify(det(Co));
fprintf(['Controllability matrix determinant: %s\n' ...
        ' (nonzero for all physically realistic vehicle parameter ranges where J_L*K_s ~= C_s*b_L)\n\n'], ...
        char(det_Co))
```

```
Controllability matrix determinant: (Km^3*(C_s*b_L - J_L*K_s))/(J_L^2*J_m^3*L^4)
(nonzero for all physically realistic vehicle parameter ranges where J_L*K_s ~= C_s*b_L)
```

```
% Desired closed-loop poles:
% -100      fast electrical mode (near natural L/R pole)
% -15       moderate real pole
% -8 +/- 8i dominant complex pair (zeta=0.71, settling ~0.5s)
p_des = [-100, -15, -8+8i, -8-8i];
fprintf(['Desired poles: -100, -15, -8+8i, -8-8i\n' ...
        ' (dominant pair: zeta=0.71, ~4% overshoot, ~0.5s settling)\n\n'])
```

```
Desired poles: -100, -15, -8+8i, -8-8i
(dominant pair: zeta=0.71, ~4% overshoot, ~0.5s settling)
```

```
% Symbolic pole placement: solve for K such that eig(A - B*K) = p_des
syms lambda
k = sym('k', [1, n]);
char_eqn = det(A - B*k - lambda*eye(n));
eqns = subs(char_eqn, lambda, p_des);
sol = solve(eqns, k);
K_sym = [sol.k1, sol.k2, sol.k3, sol.k4];
```

State feedback gains $K(J_L)$:

```
% Selective substitution: fix motor/shaft constants, keep J_L free
```

```
K_of_JL = simplify(subs(K_sym, motor_params, motor_values))
```

```
K_of_JL =
```

$$\left[\frac{47833600 J_L^2 - 4240160 J_L + 10201}{16000000 J_L^2} \quad \frac{4160 J_L - 101}{40000 J_L} \quad -\frac{33600000 J_L^2 + 2224200 J_L - 10201}{160000 J_L^2} \quad \frac{1536000}{160000 J_L^2} \right]$$

Precompensator $\bar{N}(J_L)$ for Zero Steady-State Error

Compute reference scaling so load speed tracks the setpoint. Since K and the system matrices depend on J_L , so does $\bar{N}$.

```
N_ss = [A, B; C, D] \ [zeros(n, 1); 1];
```

```
Nx = N_ss(1:n);
```

```
Nu = N_ss(end);
```

```
Nbar_sym = Nu + K_sym * Nx;
```

Reference scaling Nbar(J_L):

```
Nbar_of_JL = simplify(subs(Nbar_sym, motor_params, motor_values))
```

```
Nbar_of_JL =
```

$$\frac{48 J_L}{5}$$

Complete control law: $u = -K(J_L) \cdot x + \bar{N}(J_L) \cdot r$

4. Verification: Desired Poles Are Achieved for Any J_L

Form the closed-loop system and confirm the eigenvalues match the desired poles regardless of load inertia. Step responses show identical settling behavior across the full J_L range.

```
A_cl = A - B*K_sym;
```

```
A_cl_of_JL = simplify(subs(A_cl, motor_params, motor_values));
```

```
J_L_test = [0.01, 0.05, 0.2, 0.5];
```

```
labels = ["Lost traction", "Sedan", "SUV", "Truck"];
fprintf('Closed-loop pole verification:\n')
```

Closed-loop pole verification:

```
for idx = 1:length(J_L_test)
    poles_cl = eig(double(subs(A_cl_of_JL, J_L, J_L_test(idx))));
    poles_cl = sort(poles_cl, 'ComparisonMethod', 'real');
    fprintf(' J_L = %.2f (%s):', J_L_test(idx), labels(idx));
    for p = 1:length(poles_cl)
        fprintf(' %.1f%+.1fi', real(poles_cl(p)), imag(poles_cl(p)));
    end
    fprintf('\n');
end
```

```
J_L = 0.01 (Lost traction):
-100.0+0.0i -15.0+0.0i -8.0-8.0i -8.0+8.0i
J_L = 0.05 (Sedan):
-100.0+0.0i -15.0+0.0i -8.0-8.0i -8.0+8.0i
J_L = 0.20 (SUV):
-100.0+0.0i -15.0+0.0i -8.0-8.0i -8.0+8.0i
J_L = 0.50 (Truck):
-100.0+0.0i -15.0+0.0i -8.0-8.0i -8.0+8.0i
```

% Step responses across the inertia range

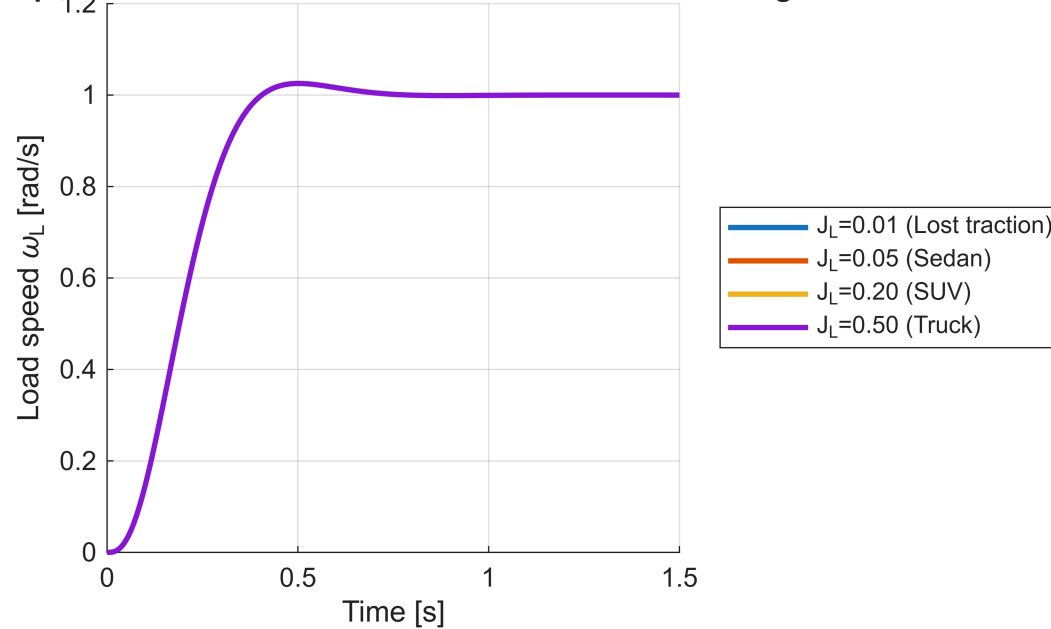
```
clf
t = 0:0.001:1.5;
colors = lines(length(J_L_test));
hold on
for idx = 1:length(J_L_test)
    J_val = J_L_test(idx);
    A_n = double(subs(A_motor, J_L, J_val));
    K_n = double(subs(K_of_JL, J_L, J_val));
    Nbar_n = double(subs(Nbar_of_JL, J_L, J_val));
    sys_cl = ss(A_n - B_Val*K_n, B_Val*Nbar_n, C_Val, D_Val);
    [y, ~] = step(sys_cl, t);
    plot(t, y, 'Color', colors(idx,:), 'LineWidth', 2, ...
        'DisplayName', sprintf('J_L=%.2f (%s)', J_val, labels(idx)))
end
legend('Location', 'eastoutside')
```

```

title('Adaptive Gains Maintain Performance Across Inertia Range')
xlabel('Time [s]'); ylabel('Load speed \omega_L [rad/s]')
grid on

```

Adaptive Gains Maintain Performance Across Inertia Range



Fixed vs. Adaptive Gains

What happens when gains designed for one J_L are used at a different J_L ? Scenario: controller tuned for low inertia / lost traction ($J_L = 0.02$), then applied when the load inertia increases to a heavy-vehicle condition ($J_L = 0.20$). The low-inertia fixed gains are too aggressive for the heavier plant and move the closed-loop poles into the right-half plane.

```

J_L_design = 0.02;
J_L_actual = 0.2;
% Fixed gains (designed for J_L_design, applied at J_L_actual)
K_fixed = double(subs(K_of_JL, J_L, J_L_design));
Nbar_fixed = double(subs(Nbar_of_JL, J_L, J_L_design));
A_actual = double(subs(A_motor, J_L, J_L_actual));
sys_fixed = ss(A_actual - B_Val*K_fixed, B_Val*Nbar_fixed, C_Val, D_Val);

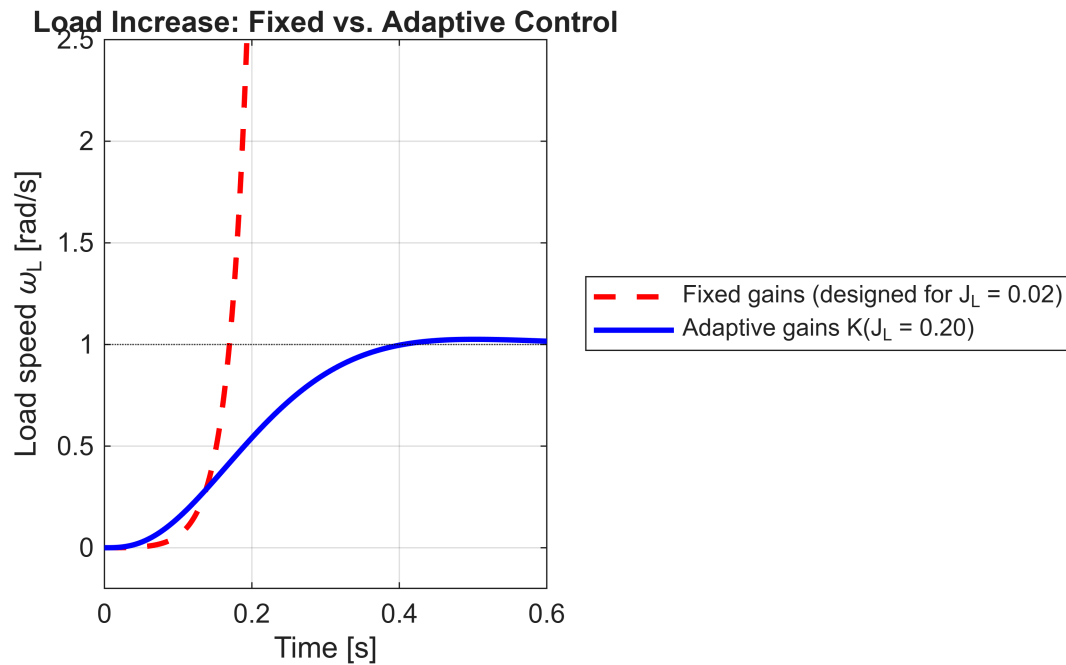
```


Fixed-gain closed-loop poles for the mismatched plant:

```
poles_fixed_mismatch = eig(A_actual - B_Val*K_fixed)
```

```
poles_fixed_mismatch = 4×1  
    38.2862  
     0.1075  
   -46.6687  
  -100.0000
```

```
% Adaptive gains (correct for J_L_actual)  
K_adapt = double(subs(K_of_JL, J_L, J_L_actual));  
Nbar_adapt = double(subs(Nbar_of_JL, J_L, J_L_actual));  
sys_adapt = ss(A_actual - B_Val*K_adapt, B_Val*Nbar_adapt, C_Val, D_Val);  
t = 0:0.0005:0.6;  
[y_fixed, ~] = step(sys_fixed, t);  
[y_adapt, ~] = step(sys_adapt, t);  
clf  
plot(t, y_fixed, 'r--', 'LineWidth', 2); hold on  
plot(t, y_adapt, 'b-', 'LineWidth', 2);  
yline(1, ':k');  
legend('Fixed gains (designed for J_L = 0.02)', ...  
       'Adaptive gains K(J_L = 0.20)', 'Location', 'eastoutside')  
title('Load Increase: Fixed vs. Adaptive Control')  
xlabel('Time [s]'); ylabel('Load speed \omega_L [rad/s]')  
ylim([-0.2 2.5])  
grid on
```



The plot limits keep the adaptive response visible while the fixed-gain response leaves the useful speed range. In practice, the armature voltage V is limited by the power electronics (e.g., 48 V), so a production controller would also include voltage limiting and anti-windup.

5. Code Generation: Deployable Controller Functions

Generate MATLAB® and C code from the symbolic expressions for use in simulation. These functions take J_L as input and return the correct gains.

```
matlabFunction(K_of_JL, 'File', 'controllerGains', 'Vars', {J_L}, ...
    'Outputs', {'K'});
matlabFunction(Nbar_of_JL, 'File', 'referenceScaling', 'Vars', {J_L}, ...
    'Outputs', {'Nbar'});
fprintf('\n--- Generated controllerGains.m ---\n')
```

```
--- Generated controllerGains.m ---
```

```
type controllerGains.m
```

```

function K = controllerGains(J_L)
%controllerGains
%    K = controllerGains(J_L)

%    This function was generated by the Symbolic Math Toolbox version 26.1.
%    11-Aug-2026 11:47:43

t2 = J_L.^2;
t3 = 1.0./t2;
K = [(t3.*(J_L.*-4.24016e+6+t2.*4.78336e+7+1.0201e+4))./1.6e+7,(J_L.*4.16e+3-1.01e+2)./(J_L.*4.0e+4),t3.*(J_L.*2.2242e+6+t2.*3.36e+7-1.0201e+4).*(-6.25e-6),t3.*(J_L.*2.2242e+6+t2.*3.36e+7-1.0201e+4).*(-6.25e-6)]./1.6e+7;
end

```

```
fprintf('\n--- Generated referenceScaling.m ---\n')
```

```
--- Generated referenceScaling.m ---
```

```
type referenceScaling.m
```

```

function Nbar = referenceScaling(J_L)
%referenceScaling
%    Nbar = referenceScaling(J_L)

%    This function was generated by the Symbolic Math Toolbox version 26.1.
%    11-Aug-2026 11:47:43

Nbar = J_L.*(4.8e+1./5.0);
end

```

```
% C code for external use
```

```
fprintf('\n--- C code for K(J_L) ---\n')
```

```
--- C code for K(J_L) ---
```

```
ccode(K_of_JL)
```

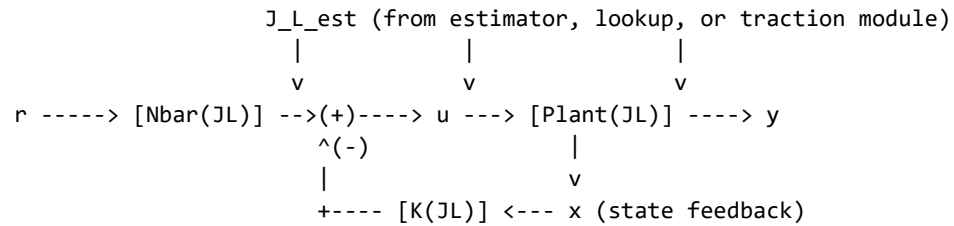
```

ans =
' K_of_JL[0][0] = (1.0/(J_L*J_L)*(J_L*-4.24016E+6+(J_L*J_L)*4.78336E+7+1.0201E+4))/1.6E+7;
  K_of_JL[0][1] = (J_L*4.16E+3-1.01E+2)/(J_L*4.0E+4);
  K_of_JL[0][2] = 1.0/(J_L*J_L)*(J_L*2.2242E+6+(J_L*J_L)*3.36E+7-1.0201E+4)*(-6.25E-6);
  K_of_JL[0][3] = (1.0/(J_L*J_L)*(J_L*4.268462E+8-(J_L*J_L)*5.05472E+9+(J_L*J_L*J_L)*1.536E+10-1.030301E+6))/1.6E+9; '

```

6. Simulink® Integration

The generated functions can be called directly from MATLAB Function blocks in Simulink. The model architecture:



- controllerGains(J_L) computes the 1x4 state feedback gain vector K
- referenceScaling(J_L) computes the precompensator Nbar
- J_L can come from a lookup table, an online estimator, or the traction control module

Run buildSimulinkModel.m to create a working Simulink model that demonstrates this architecture with a traction-loss scenario (J_L switches from 0.2 to 0.02 at $t = 5$ seconds).

For direct Simulink block generation: matlabFunctionBlock('model/block', K_of_JL, 'Vars', {J_L})